**Vukašin Đuričić**
**SR 08/2023**

# Conway's Game of Life

***performance optimization report***

## Introduction

- This report presents a comparative analysis of Conway's Game of Life, a cellular automaton simulation, implemented across two programming languages: **Python** and **Rust.** The main objective is to evaluate performance differences between sequential and parallel implementation in both languages.

To provide context for performance metrics, the experiments were done on specific hardware and software environment. The technical specifications of the system used for all benchmarks are detailed below.

## System architecture and Specifications

**Processor (CPU) :**

- Model: **Intel® Core™ i5-7300U CPU**
- Base frequency: **2.60 GHz**
- Maximum frequency (Turbo): **~3.50 GHz**
- Architecture: **2 Physical Cores, 4 Logical Processor (Hyper-Threading enabled)**
- NUMA organization: **1 node**

**Cache memory Hierarchy:**

- L1 cache: **128 KiB total (64 KiB Data + 64 KiB Instruction)**
- L2 cache: **512 KiB (256 KiB per core)**
- L3 cache: **3 MiB (Shared)**

**System Memory (RAM):**

- Capacity: **16 GB**
- Type: **DDR4**
- Speed: **2133 MT/s**

**Operating System:**

- **Linux Mint 22.3 (Zena)**

# Game of Life parallelization in theory

- By the nature of Conway's game of Life, the problem is classified as "Data parallel", which means that the parallelization of the problem requires doing the same operations on different parts of the data set which scales really good with many cores. The calculation of a single cell's future state is independent of the calculation of other cells within the same generation. So the computations are mostly parallelizable. (around 95-99%)

The sequential portion is limited to (minimally):

- memory allocation, creating a new matrix in memory
- thread synchronization barriers between generations, before starting generation 2 all threads must finish their work at generation 1

# Python implementation

**Libraries used:**

- **Python 3.12.3**
- **Numba 0.63.1**
- **NumPy 2.3.5**

**Why NumPy?**

- Regular python lists are represented in memory as arrays of pointers to concrete values, where the values are scattered around the RAM, NumPy keeps the elements as a contiguous block of concrete values. By going for this approach we introduce Cache locality. By accessing a cell from the previous generation grid other than caching the accessed cell, we also cache it's adjacent cells in super fast L1, L2 cache which make the writes and reads drastically faster. Python integers are objects with their own methods so they take up quite some space (28 bytes), this problem needs only 0/1 values so NumPy *np.int8* is more than enough.

**Why Numba?**

- Python is an interpreted, dynamically typed language which is way slower in comparison to compiled, statically typed languages such as Rust or C, because of type checking overhead which is executed in runtime. Numba solves pythons performance issues by taking python blocks of code and translates them directly into machine code by using LLVM (Low Level Virtual Machine), which makes annotated functions execution time similar as if they were written in C. Helps us avoid the python's Global interpreter lock which forbids parallel execution of tasks to avoid race conditions since most python features are not thread safe and don't use any kind of locking mechanism. GIL runs parallel code as sequential on a single python thread. Also introduces simple syntax by using prange, to run loop tasks in parallel which I am sure was an overkill for this project but was a relief to work with :D

## Effective serial/parallel portion of code

- To determine the portion of the code that is sequential, we used the Karp-Flatt metric which is a variation of Amdahl's Law, which uses measured speed up to calculate the effective sequential portion of code in systems.

First we calculate the speed up:

$$S = Tseq / Tpar,$$

where S represents speed up, T seq is sequential implementation execution time and T par is parallel implementation execution time.

From this the sequential portion (s) is derived using the following formula:

$$s = ((n/S) – 1) / (n – 1),$$

Where n represents the number of cores the parallel run was executed on.

- Since the machine I was running benchmarks on has 2 physical cores and 2 virtual with hyper-threading enabled, the maximum amount of threads I could utilize was 4. After running sequential, 2, 3 and 4 cores the results were the following:

**Execution time for different configurations**

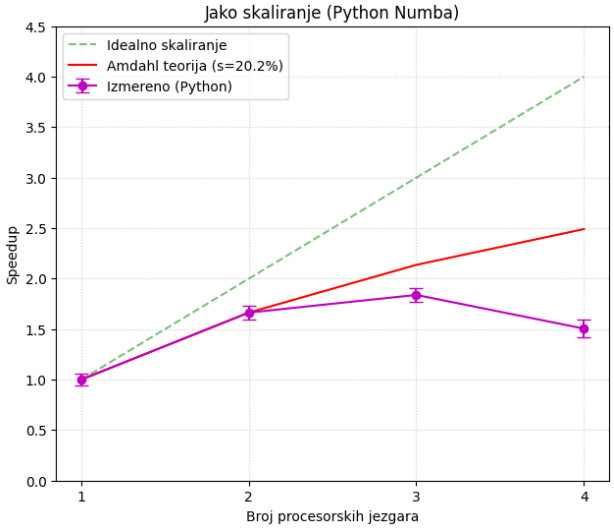
| Configuration | Execution time (ms) |
|---|---|
| 1 thread | 69 |
| 2 threads | 37 |
| 3 threads | 35 |
| 4 threads | 30 |

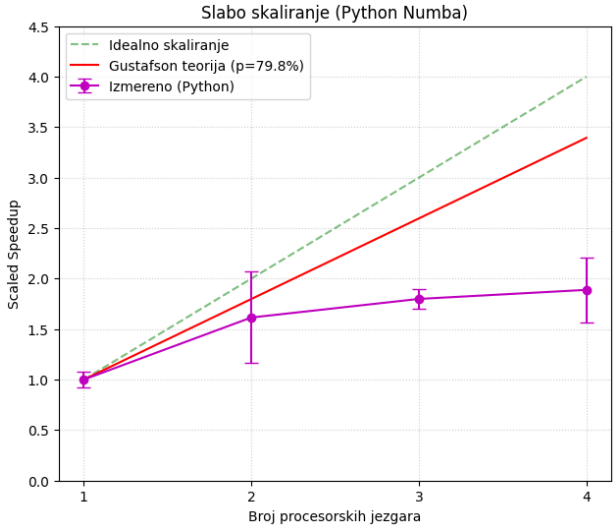
**Calculation of the sequential fraction**

| No. of threads | Speed up | Percentage |
|---|---|---|
| 2 | 1.86x | 7.5% |
| 3 | 1.97x | 26.0% |
| 4 | 2.30x | 24.7% |

Where the parallel fraction of the code is calculated as 100% - sequential fraction percentage.

## Hard vs weak scaling




**Strong scaling**

| No. of threads | Mean (s) | Std. Dev. (s) | Outliers (s) |
|---|---|---|---|
| 1 | 0.0600 | 0.0035 | 0.0584, 0.0733, 0.0731, 0.0750, 0.0722 |
| 2 | 0.0364 | 0.0044 | 0.0473, 0.0477, 0.0483, 0.0479, 0.0479 |
| 3 | 0.0324 | 0.0045 | 0.05 |
| 4 | 0.04 | 0.0055 | 0.0582, 0.0606 |

**Weak scaling**

| No. of threads | Mean (s) | Std. Dev. (s) | Outliers (s) |
|---|---|---|---|
| 1 | 0.05 | 0.0036 | 0.057, 0.053 |
| 2 | 0.0625 | 0.0194 | 0.1531, 0.0945 |
| 3 | 0.0833 | 0.02 | 0.0941, 0.0833 |
| 4 | 0.1053 | 0.02 | 0.1116, 0.1183, 0.1627 |

# Rust Implementation

**Libraries used:**

- **Rust 1.19.1**
- **Rayon 1.10**

**Why Rayon?**

- Manually spawning threads for every parallel task incurs significant initialization overhead due to OS-level resource allocation. Rayon solves this performance bottleneck by

initializing a global thread pool once and reusing these threads throughout the program's life cycle. This approach completely eliminates the cost of constantly creating and destroying threads. Rayon converts sequential iterators into parallel ones using a smart work-stealing algorithm, ensuring that idle cores in the pool automatically take tasks from busy ones. This guarantees maximum CPU utilization and data-race freedom at compile time, making .par_chunks_exact_mut() a highly efficient drop-in replacement for standard loops without the penalty of thread management.

## Effective serial/parallel portion of code

- The explanation on how we calculate the serial/parallel portion of code is written in the python section so I won't repeat it.

**Execution time for different configurations**

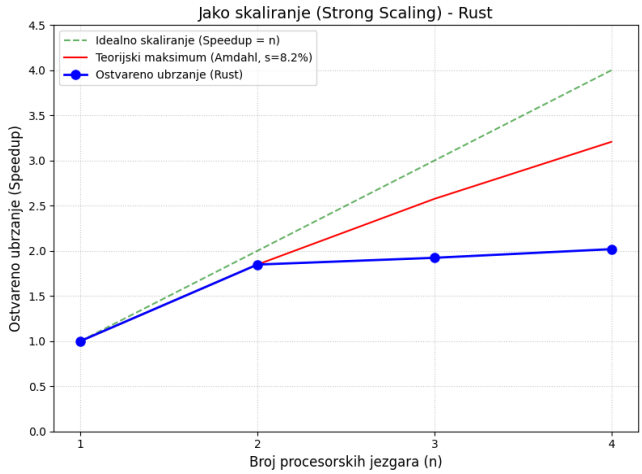
| Configuration | Execution time (ms) |
|---|---|
| 1 thread | 60 |
| 2 threads | 38 |
| 3 threads | 32 |
| 4 threads | 35 |

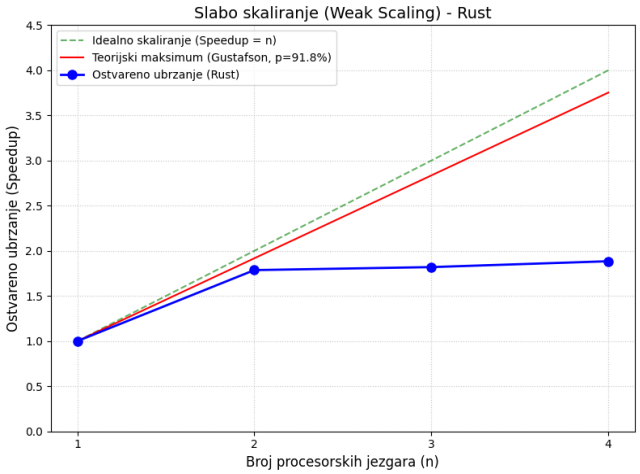
**Calculation of the sequential fraction**

| No. of threads | Speed up | Percentage |
|---|---|---|
| 2 | 1.58x | 26.7% |
| 3 | 1.88x | 30% |
| 4 | 1.7x | 44.4% |

Where the parallel fraction of the code is calculated as 100% - sequential fraction percentage.

## Hard vs weak scaling



**Strong scaling**

| No. of threads | Mean (s) | Std. Dev. (s) | Outliers (s) |
|---|---|---|---|
| 1 | 2.03 | 0.25 | 2.506, 2.862, 2.464, 2.442 |
| 2 | 1.100 | 0.02 | 1.12 |
| 3 | 1.06 | 0.02 | 1.064, 1.085 |
| 4 | 1.015 | 0.03 | 1.09 |

**Weak scaling**

| No. of threads | Mean (s) | Std. Dev. (s) | Outliers (s) |
|---|---|---|---|
| 1 | 1.76 | 0.03 | 1.823, 1.806, 1.879, 1.813 |
| 2 | 1.97 | 0.05 | 1.869, 2.135 |
| 3 | 2.9 | 0.03 | 3.02 |
| 4 | 3.73 | 0.08 | 4.024, 3.827, 3.818, 3.857 |